27. Suposem que implementem una seqüència d'n operacions sobre una estructura de dades on la i-èsima operació té un cost i, si i és una potència de 2, altrament el cost és 1. Determineu el cost amortitzat per operació utilitzant el mètode de l'estalviador (accounting).

Solució Sigui c_i el cost de la operació i,

$$c_i = \begin{cases} i, & \text{si } i \text{ es una potencia exacta de 2} \\ 1 & \text{altrament.} \end{cases}$$

Carreguem per cada operació un cost de $3 = \hat{c}_i$. Aleshores si i no és un potencia exacta de 2, paguem 1 i estalviem 2. Si $i = 2^k$ paguem i utilitzant credit enmagazemat.

Operació	cost	cost real	credit que queda
1	3	1	2
2	3	2	3
3	3	1	5
4	3	4	4
5	3	1	6
6	3	1	8
7	3	1	10
8	3	8	5

Com per a cada i $\hat{c}_i = 3$, $\sum_{i=1}^{n} \hat{c}_i = 3n$. Per tant $\sum_{i=1}^{n} \hat{c}_i \geq \sum_{i=1}^{n} c_i \Rightarrow$ credit= cost amortitzat -cost real ≥ 0.

Com el cost amortitzat de cada operació és $O(1)$, i la quantitat de credit sempre és > 0, el cost total de les n operacions és $O(n)$.

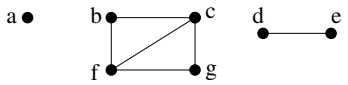
28. Sigui S una estructura de dades que es una pila (stack), amb tres operacions: Les dues genèriques **Push** (S, x), que col·loca x al cim de la pila i **Pop** (S) que retorna l'objecte x que es al cim de la pila. Considerem una nova operació **Multipop** (S, k), que donat una pila amb mes de $k \geq 0$ elements retorna els k primers elements. Per ex. Si $S = [23, 17, 6, 39, 10, 47]$ aleshores **Multipop** $(S, 4)$, retornara seqüencialment els elements 23,17,6 i 39, i deixara la pila $[10, 47]$. Si ara tornéssim a fer **Multipop** $(S, 4)$. Retornaria 10 i 47 i deixaria buida la pila. Notem que el cost cas pitjor de **Multipop** (S, k) on S conte n elements, es $= \min\{n, k\}$.
Si considerem una seqüencia de n operacions **Push, Pup, Multipop** sobre una pila que inicialment es buida, el **cas pitjo**r de **Multipop** pot ser $O(n)$ (per ex. fem $n/2$ **Push** a S i després fem **Multipop** $(S, n/2)$. Per tant, el cas pitjor de fer **Multipop** sobre n elements en $O(n^2)$.
Demostreu que utilitzant anàlisi amortitzant podem obtenir un millor cost amortitzat, fent servir:

(a) el mètode d'agregació.

(b) el mètode de l'estalviador (accounting).

29. Donat un multiconjunt dinàmic d'enters, S, que pot tenir valors duplicats, dissenyeu una estructura de dades que done suport a les següents operacions: INSERT(S, x), insereix un enter x a S, DELETE-LARGER-HALF(S), elimina els $\lfloor |S|/2 \rfloor$ elements més grans de S.
Expliqueu com implementaríeu aquesta estructura de dades de manera que qualsevol seqüència de m operacions INSERT i DELETE-LARGER-HALF, es pugui fer en $O(m)$ passos. La vostra implemenatació també ha d'incloure una manera de treure els elements de S en $O(|S|)$ passos.

30. Donat un graf no dirigit $G = (V, E)$ amb n vèrtexs i m arestes, dissenyeu un algorisme que utilitzant UNION-FIND (tal com s'ha explicat a classe) ens digui si G és o no connex i en cas de que no ho sigui, puguem averigus a quina component connexa pertany un vèrtex v donat. Quina és la complexitat del vostre algorisme?. Apliqueu el vostre algorisme al següent exemple



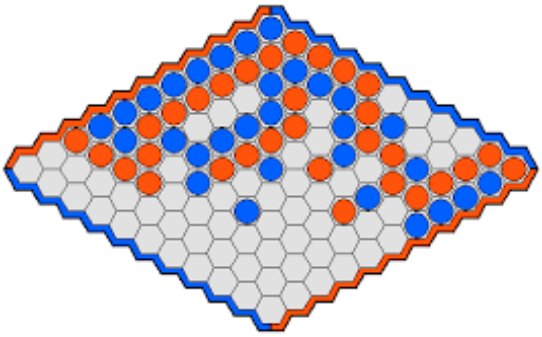
El problema de trobar la connectivitat dún graf no dirigit té complexitat $O(n + m)$ utilitzant DFS o BFS, en quins casos UNION-FIND és més ràpid?

31. Tenim 20 elements i fem un MakeSet. Suposem que els conjunts resultants estan enumerats de 0 fins a 19. Executem el següent codi:

```
for i = 0 to 14 do
    union(find(i), find(i + 5))
end for
for i = 15 to 17 do
    union(find(i), find(i + 2))
end for
```

Dibuixeu la seqüencia d'arbres fins a arribar al resultat final (dos arbres) quan la implementació utilitza unió per rang amb compressió de camins. Doneu el rang dels nusos al dos arbres resultants.

32. El joc d'HEX té com un dels seus inventors, al matemàtic John Nash. En aquest joc, dos jugadors, un amb color negre i l'altre amb color e blanc, fan torns on a cada torn el jugador que li toca col·loca una pedra del seu colors a una posició encara buida, a una xarxa $n \times n$ de cel.les hexagonals. Un cop col·locada una pedra, no es pot moure. L'objectiu de cada jugador és connectar els costats del mateix color a la graella, amb un camí continuo fet amb les seves pedres. Dues cel·les es consideren connectades si comparteixen una vora les dues tenen la pedres amb el mateix color. Descriviu un esquema eficient que determini, després de cada jugada, si el jugador que acaba de jugar ha guanyat el joc d'HEX.



Red won

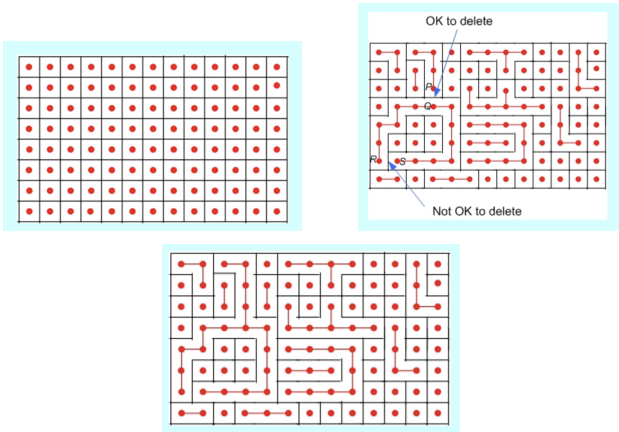
33. Una de les tasques d'un sistema operatiu és la feina de planificar els càlculs per ser realitzats pels processadors que formen part d'aquest sistema. Una subtasca que apareix sovint és la de planificar l'execució d'una seqüència σ de $O(n)$ operacions sobre una cua de prioritat. Cada operació a σ és una inserció `insert(i)`, per algun $1 \leq i \leq n$, o un esborrat `removeMin()`. El problema es coneix com el problema *offline-min*, ja que tota la seqüència, σ, és part de l'entrada.

    Sorprenentment, la naturalesa d'aquest problema ens dóna una manera més ràpida de fer les operacions en σ que utilitzar directament un heap. Observem que si k és el número més petit inserit a la cua en arribar a algun punt de σ, llavors sabem que el següent `removeMin()` en σ rebrà k. De la mateixa manera, després d'haver tret l'inserció de k i el corresponent `removeMin()` de σ, podem repetir aquest argument sobre les operacions que queden. Utilitzeu aquesta observació per dissenyar un algorisme per a fer totes les operacions en σ en $O(|\sigma| + n\beta(n))$, i, per tant, resoldre el problema offline-min eficientment, on $\beta(n)$ és el millor cost amortitzat per operació de union-find.

34. Considereu el següent algoritme de generació de laberint aleatori basat en una graella en el que ens donen dos cel.las x e y a les cantonades.

Comenceu amb cada vèrtex a la xarxa $n \times n$ formant el seu propi conjunt
identificats amb números en $[0, n^2 - 1]$
Crea totes les parets interiors i exteriors
while x i y no estan al mateix conjunt
Trieu una paret interior a l'atzar
if els vèrtex adjacents a aquesta paret estan en diferents conjunts
traieu la paret i feu una UNIO
endif
endwhile

A la figura següènt teniu un exemple de l'estat inicial. Seguit d'una il.lustració de casos en els que el if intermedi és cert/fals i del resultat desprès de la UNIO en el cas en que es pot..



Expliqueu el següent:

(a) Què representa un conjunt a mida que l'algorisme progressa?

(b) Per què no deixem de realitzar UNION una vegada que la cèl·la inicial i la cel·la final estan en la mateixa classe d'equivalència?

(c) Com garanteix l'algorisme que només hi haurà una ruta des de la cel·la inicial fins a la cel·la final?

(d) Exactament quants UNION es duen a terme? Expliqueu la vostra resposta.

35. **(Revisió)** Un professor rep n sol·licituds de revisions d'examen. Abans de començar, el professor mira la llista dels n estudiants que han sol·licitat revisió i pot calcular, per a cada estudiant i, el temps t_i que utilitzarà per atendre l'i-èsim estudiant. Per a estudiant i, el temps d'espera e_i és el temps que el professor triga a revisar els exàmens dels estudiants que fan la revisió abans que i.

Dissenyeu un algorisme per a computar l'ordre en que s'han de revisar els exàmens dels n estudiants de manera que es minimitzi el temps total d'espera: $T = \sum_{i=1}^{n} e_i$.

Solució

L'algorisme ordena els estudiants en ordre creixent de t_i.

Per veure que aquesta ordenació és òptima farem servir un argument d'intercanvi. Suposem que els estudiants estan ordenats de forma òptima y que aquesta ordenació no és la nostre. Llavors hem de tenir un estudiant i pel què $t_i > t_{i+1}$. Si calculem el temps de espera de i i de $i+1$ tenim

$$e_i = \sum_{j<i} t_j \text{ i } e_{i+1} = \sum_{j<i+1} t_j = e_i + t_j.$$

Si intercanvien la posició de l'estudiant i amb la del $i+1$ els temps d'espera dels estudiants amb $j < i$ o $j > i+1$ no canviam (d'acord amb la definició). Després del intercanvi el temps d'espera del estudiant $i+1$ és e_i i el del estudiant $i+1$ és $e_i + t_{i+1}$. Si sumen aquest temps tenim

$$e_i + e_i + t_{i+1} < e_i + e_i + t_i = e_i + e_{i+1}.$$

Per tant el temps d'espera total ha millorat i en conseqüència arribem a que l'ordenació no és òptima.

36. Tenim un graf no dirigit $G = (V, E)$. Donat un subconjunt $V' \subseteq V$ el subgraph induït per V' és el graf $G[V'] = (V', E')$ on $E' = E \cap (V' \times V')$. El grau d'un vèrtex a un graf és el nombre d'arestes incidents al vèrtex. Doneu un algorisme eficient per al següent problema: donat G i un enter positiu k, trobar el subconjunt (si hi ha algun) més gran V' de V, tal que cada vèrtex a V' té grau $\geq k$ a $G[V']$.

37. El problema de la *partició interval* (*Interval Partitioning Problem*) és similar al problema de la selecció d'activitats vist a classe però, en lloc de tenir un únic recurs, tenim molts recursos (és a dir, diverses còpies del mateix recurs). Doneu un algorisme que permeti programar totes les activitats fent servir el menor número possible de recursos.

38. **Streaming.** Tenim n paquets de vídeo que s'han d'enviar seqüencialment per una única línia de comunicació (això s'anomena *streaming*). El paquet i-èsim té una grandària de b_i bits i triga t_i segons a travessar, on t_i i b_i són enters positius (no es poden enviar dos paquets al mateix temps). Hem de decidir una planificació de l'ordre en què hem d'enviar els paquets de manera que, un cop tenim un ordre, no pot haver-hi retard entre la fi d'un paquet i el començament del següent. Assumirem que comencem a l'instant 0 i finalitzem al $\sum_{i=1}^{n} t_i$. A més, el proveïdor de la connexió vol utilitzar una amplada de banda no massa gran, per tant s'afegeix la restricció següent: Per a cada enter $t > 0$, el nombre total de bits que enviem de temps 0 a t ha de ser $\leq rt$, per a una $r > 0$ fixada (noteu que aquesta restricció no diu res per a períodes de temps que no comencen a l'instant 0). Direm que una planificació és *vàlida* si satisfà la restricció prèvia.

El problema a resoldre és el següent: donat un conjunt de n paquets, cadascun especificat per la seva grandària b_i i la durada de la seva transmissió t_i, i donat el valor del paràmetre r, hem de determinar si existeix una planificació vàlida dels n paquets. Per exemple, si $n = 3$ amb $(b_1, t_1) = (2000, 1)$, $(b_2, t_2) = (6000, 2)$ i $(b_3, t_3) = (2000, 1)$ amb $r = 5000$, aleshores la planificació $1, 2, 3$ és vàlida, donat que compleix la restricció.

Per a resoldre aquest problema, heu de resoldre els apartats següents:

(a) Demostreu o doneu un contraexemple a l'enunciat de: és veritat que existeix una planificació vàlida si, i únicament si, per a cada paquet i tenim $b_i \leq rt_i$.

(b) Doneu un algorisme que per a una entrada de n paquets (cadascun especificat per (b_i, t_i)) i el paràmetre r, determini si existeix una planificació vàlida. El vostre algorisme hauria de tenir complexitat polinòmica en n.

39. Sigui $G = (V, E)$ un graf no dirigit. Un subconjunt $C \subseteq V$ s'anomena *recobriment de vèrtexs* de G si

$$\forall \{u, v\} \in E : \{u, v\} \cap C \neq \emptyset.$$

Donat $G = (V, E)$, un recobriment de vèrtexs C és diu que és *minimal* si per qualsevol $C' \subseteq V$ a G, tal que $C' \subset C$ hem de tenir que C' no és un recobriment de vèrtexs.

Un conjunt $C \subset V$ és un *recobriment de vèrtexs mínim* si C és un recobriment de vèrtexs a G amb mínima cardinalitat. (Quan G és un arbre, hi ha un algorisme polinòmic per a trobar un recobriment de vèrtexs mínim a G, però per a G generals el problema és NP-hard).

En aquest problema demostrareu que, a diferència del problema de trobar un recobriment de vèrtexs mínim, el problema de trobar un recobriment de vèrtexs minimal pot ser resolt en temps polinòmic.

(a) Demostreu que un recobriment de vèrtexs minimal no necessàriament ha de ser un recobriment de vèrtexs mínim.

(b) Demostreu que tot recobriment de vèrtexs mínim també és minimal.

(c) Doneu un algorisme polinòmic per trobar un recobriment de vèrtexs minimal a G.

40. CoopC és una companyia de lloguer de cotxes que vol diversificar el seu negoci per tal de competir en els desplaçaments curts. La companyia té oficines distribuïdes a Barcelona i un dipòsit central de cotxes al Prat. CoopC vol un mètode que li permeti processar les peticions de trajectes curts rebuts pel dia següent. Per això, vol determinar el nombre de cotxes que ha de deixar a cada oficina a l'inici del dia dedicats a trajectes curts. Aquesta assignació ha de permetre tenir suficients cotxes disponibles al llarg del dia (en cadascuna de les oficines) per tal de poder cobrir tots els trajectes curts del dia. A més, CoopC vol minimitzar el nombre de cotxes destinats durant el dia a desplaçaments curts.

    La informació de la qual disposa CoopC per a cada sol·licitud de trajecte curt per al proper dia és la següent: una tupla (l, t, l', t') on el parell (l, t) indica l'oficina i l'hora de recollida del vehicle i el parell (l', t') indica l'oficina i l'hora de devolució.

    A efecte d'aquesta aproximació CoopC assumeix que tots els cotxes són idèntics i que disposa d'una flota prou gran per a poder cobrir qualsevol nivell de demanda de trajectes curts. A més, assumeix també que els cotxes seran recollits i retornats puntualment a les hores i locals estipulats.

    Dissenyeu un algorisme voraç que permeti obtenir el nombre de cotxes que s'han d'assignar a cada oficina, de manera que, al llarg del dia, sempre hi hagi almenys un cotxe disponible en una oficina a l'hora en què algun client l'ha de recollir d'allà. Tenint en compte que mantenir immobilitzat un cotxe té un cost alt, l'assignació obtinguda ha de garantir el requisit de disponibilitat i ha d'assignar el menor nombre possible de cotxes a cada oficina.

41. **(Vídeo blocs)** Tenim n blocs de vídeo que s'han d'enviar per una única línia de comunicació a mida que es filmen. El bloc i-èsim té una grandària de b_i bits i està disponible per iniciar la seva transmissió a temps s_i, on s_i i b_i són enters positius. Podeu assumir que transmetre un bit requereix una unitat de temps. A més, la transmissió del paquet i no pot començar abans de l'instant de temps s_i, però sí després.

Encara que no es poden enviar dos paquets al mateix temps, la transmissió d'un bloc de vídeo es pot suspendre a qualsevol instant per completar-la més endavant. Per exemple, si tenim dos blocs de vídeo amb $b_1 = 5$, $s_1 = 2$, $b_2 = 3$ i $s_2 = 0$ podríem enviar a temps 0 els 3 bits del segon paquet i a temps 3 el 5 bits del bloc 1 o bé, enviar a temps 0, 2 bits del bloc 2, desprès els 5 bits del bloc 1 i finalitzar amb el bit restant del bloc 2. També seria possible enviar a temps 0, 2 bits del bloc 2, després 2 bits del bloc 1, finalitzar el bit restant del bloc 2, i finalitzar els bits del bloc 1. Observeu que no podem començar la transmissió de cap bit del bloc 1 fins al instant 2.

Una vegada fixada la planificació de l'ordre en què enviarem els bits dels diferents blocs, ens interessa el temps en què finalitza la transmissió de cada bloc. Anomenem f_i al temps en què finalitza la transmissió del bloc i. Observeu que, per a l'exemple anterior, la primera planificació dona $f_1 = 8$ i $f_2 = 3$, la segona planificació dona $f_1 = 7$ i $f_2 = 8$, mentre que la tercera dona $f_1 = 8$ i $f_2 = 5$.

Volem obtenir una planificació de la transmissió dels n blocs de vídeo que minimitzi la suma dels temps de finalització de la transmissió dels blocs, és a dir $\sum_{i=1}^{n} f_i$. Doneu un algorisme que, per a una entrada de n blocs, cadascun especificat per (b_i, s_i), determini una planificació (començant a temps 0) resolgui el problema proposat.

Solució: Voy a utilizar un algoritmo voraz siguiendo la siguiente regla: En cada instante de tiempo programamos el envío de un bit del paquete disponible al que le quedan menos bits por finalizar la transmisión.

Observemos que si una solución óptima no sigue esta regla buscamos el primer instante de tiempo t, en esta planificación, en que se envía un bit de un paquete i al que le faltan m_i bits por transmitir mientras que hay un paquete j disponibles a tiempo t al que le le faltan m_j bits por transmitir a tiempo t con $m_j < m_i$. Consideramos ahora los instantes de tiempo, $\geq t$, en que se envian los bits de i y de j. Podemos iniciar la transmisión de j a tiempo t, retrasando todos sus bits, al tiempo del bit de j previo. Compensar, retrasando la transmisión de los bits de i hasta llegar a utilizar el espacio dejado por el último buiit de j. La nueva planificación es válida y proporciona nuevos tiempos de finalización f'_i, f'_j. Si $f_j < f_i$, al replanificar los paquetes tenemos $f'_j < f_j$ y $f'_i = f'_j$. En este caso la planificación no seria óptima. Si $f_j > f_i$, $f'_j = f_i$ y $f'_i = f'_j$, la suma no cambia, por lo que la nueva planificación sigue siendo óptima. Realizando los intercambios correspondientes podemos conseguir una planificación óptima con el criterio de nuestro algoritmo voraz.

Para implementar el algoritmo voraz tenemos que ir con cuidado para que el número de pasos sea polinómico en el número de bloques y no en función del tiempo de finalización de la planificación que es función de los valores de los números s_i i t_i.

Para determinar eficientemente los paquetes activos ordenamos los paquetes en orden creciente de s_i, en caso de empate por orden creciente de b_i. Mantendremos una cola de prioridad, con clave el número de bits que faltan por enviar, de los paquetes disponibles y cuya transmisión no ha terminado. Por otra parte mantendremos dos índices de vídeos v_a y v_s (actual y siguiente), tres contadores de tiempo, t tiempo actual, t_a tiempo previsto finalización v_a, t_s tiempo en que v_s estará disponible.

Inicialmente $t = t_a = t_s = s_1$, $v_s = 1$, $v_a = 0$. Iremos avanzando el tiempo de acuerdo con diferentes eventos:

- Si $t_a = t_s$, introducimos en la cola los paquetes i con $s_i = t_s$ con clave b_i. v_s será el primer paquete no introducido en la cola y $t_s = s_{v_s}$.

  Extraemos el min de la cola en v_a incrementamos t_a con el número de bits que faltan de transmitir de v_a.

- Si $t_a < t_s$, planificamos los bits que faltan v_a entre t y t_a, $t = t_a$.

  Si la cola no está vacía extraemos el min de la cola en v_a, actualizamos t_a con t más el número de bits que faltan de transmitir de v_a.

  Si la cola está vacía, actualizamos t y t_a con el valor de t_s.
- Si $t_a > t_s$, transmitimos bits de v_a en los tiempos $t, \ldots, t_s - 1$, introducimos v_a en la cola con clave el número de bits que falten por transmitir.

  Actualizamos los contadores de tiempo, t y t_a para que coincidan con t_s.

La implementación es correcta ya que la prioridad en la cola solo se puede alterar cuando tenemos un nuevo vídeo disponible. Observemos que un cada paquete de vídeo entra al menos una vez en la cola. Cuando reintroducimos de nuevo un paquete, lo hacemos coincidiendo con la introducción de un paquete nuevo, pero en este caso solo reintroducimos un paquete. Esto nos da un total de $O(n)$ inserciones en la cola de prioridad.

El coste total del algoritmo es $O(n \log n)$: la ordenación inicial $O(n \log n)$y las n insercciones en la cola de prioridad con coste $O(n \log n)$.

42. Un *bottleneck spanning tree* T d'un graf no dirigit i ponderat $G = (V, E, w)$, on $w : E \to \mathbb{R}^+$, és un arbre d'expansió de G on el pes més gran és mínim sobre tots els arbres d'expansió de G. Diem que el valor d 'un bottleneck spanning tree és el pes de la aresta de pes màxim a T.

   (a) Demostreu la correctesa o trobeu un contraexemple pels enunciats següents:

      - *Un bottleneck spanning tree és també un arbre d'expansió mínim.*
      - *Un arbre d'expansió mínim és també un bottleneck spanning tree.*

   (b) Doneu un algorisme amb cost $O(|V| + |E|)$ que donat un graf G i un enter b, determini si el valor d 'un bottleneck spanning tree és $\leq b$.

43. Demostreu que un graf G té un únic MST si, per a tot tall C de G, existeix una única aresta $e \in C$ amb valor mínim. Demostreu que el el reciproc no és cert, i.e. pot ser el cas de que per un o més talls C tinguem més d'una aresta mínim pes, però que el MST sigui únic

44. **Clustering.** Una versió senzilla del *problema de l'agrupació (clustering)* és la següent. Com a entrada al problema tenim: un conjunt de n objectes $X = \{x_1, x_2, \ldots, x_n\}$, una distància $d(x_i, x_j) > 0$ per a cada parell i, j amb $i \neq j$, una $k \in \mathbb{N}$. El problema consisteix a particionar X en k subconjunts no buits $\mathcal{C} = \{C_1, C_2, \ldots, C_k\}$ (anomenats clústers) de manera que es maximitzi la separació entre clústers. La separació entre clústers a $\mathcal{C}$ és la distància mínima entre tots els possibles parells de punts a diferents clústers de $\mathcal{C}$. Doneu un algorisme polinòmic per solucionar aquest problema.

45. El centre de documentació de la UE gestiona el procés de traducció de documents pels membres del parlament europeu. En total han de treballar amb un conjunt de n idiomes. El centre ha de gestionar la traducció de documents escrits en un idioma a tota la resta d'idiomes.

Per fer les traduccions poden contractar traductors. Cada traductor està especialitzat en dos idiomes diferents; és a dir, cada traductor pot traduir un text en un dels dos idiomes que domina a l'altre, i viceversa. Cada traductor té un cost de contractació no negatiu (alguns poden treballar gratis).

Malauradament, el pressupost per a traduccions és massa petit per contractar un traductor per a cada parell d'idiomes. Per tal d'optimitzar la despesa, n'hi hauria prou en establir cadenes de traductors; per exemple: un traductor anglès $\leftrightarrow$ català i un català $\leftrightarrow$ francès, permetria traduir un text de l'anglès al francès, i del francès a l'anglès. Així, l'objectiu és contractar un conjunt de traductors que permetessin la traducció entre tots els parells dels n idiomes de la UE, amb cost total de contractació mínim.

El matemàtic del centre els hi ha suggerit que ho poden modelitzar com un problema en un graf amb pesos $G = (V, E, w)$. G té un node $v \in V$ per a cada idioma i una aresta $(u, v) \in E$ per a cada traductor (entre els idiomes u i v de la seva especialització); el pes de cada aresta seria el cost de contractació del traductor en qüestió. En aquest model, un subconjunt de traductors $S \subseteq E$ permet portar a terme la feina si al subgraf $G_s = (V, S)$ hi ha un camí entre tot parell de vèrtexs $u, v \in V$; en aquest cas direm que S és una *selecció vàlida*. Aleshores, d'entre totes les seleccions vàlides han de triar una amb cost mínim.

(a) Demostreu que quan S és una selecció vàlida de cost mínim, $G_s = (V, S)$ no té cicles.

(b) Proporcioneu un algorisme eficient per a resoldre el problema. Justifiqueu la seva correctesa i el seu cost.

Solució:

(a) Supngamos que $G_s = (V, S)$ es una selección válida de coste mínimo que tiene ciclos. Si eliminamos una arista (u, v) de un ciclo en $G_s = (V, S)$ seguimos teniendo caminos entre todos los vértices, ya que podemos ir de u a v a través de lo que queda del ciclo.

Como la selección tiene coste mínimo, y eliminando una arista de un ciclo también es solución. Tenemos que todas las aristas de un ciclo tienen coste 0. Así, mientras tengamos ciclos vamos eliminando una arista de peso 0 del ciclo. Hasta que tengamos una selección válida con coste mínimo sin ciclos.

(b) Por el aparatdo a) nos basta con buscar un árbol con peso mínimo que cubra todos los idiomas. Es decir tenemos que obtener un MST del grafo. Utilizando el algoritmo de Prim, podemos encontrarlo en tiempo $O(n \log m)$

46. Tenim un tauler de dimensions $n \times n$, amb n fitxes col·locades a certes posicions $(x_1, y_1), \ldots, (x_n, y_n)$ i una fila i. Volem determinar el mínim nombre de moviments necessaris per a posar les n fitxes a la fila i (una a cada casella). Els moviments permesos són: cap a la dreta, esquerra, amunt i avall. Durant aquests moviments es poden apilar tantes fitxes a la mateixa posició com calgui. Pero en finalitzar ha de quedar una fitxa per casella.

    Pista: El nombre de moviments verticals (amunt/avall) necessaris es pot calcular fàcilment.

47. La cadena de sota és el títol d'una cançó codificat amb codis de Huffman.

0011000101111101100111011101100000100111010010101

Donades les freqüències de les lletres que es donen a la taula de sota, obtingueu els codis de Huffmann i feu-los servir per decodificar el títol.

lletra	a	h	v	w	' '	e	t	l	o
freqüència	1	1	1	1	2	2	2	3	3

Tingueu en compte que, quan hi ha múltiples eleccions, cal un criteri de desempat. Per això podeu suposar que quan es fusionan dos arbres es posa a la esquerra el que té, la fulla de més a la esquerra, més a l'esquerra a l'ordre inicial donat a la taula. A més s'assigna a la branca esquerra el símbol 0 i a la dreta el símbol 1.

48. Tenim un alfabet Σ on per a cada símbol $a \in \Sigma$, p_a es la probabilitat que aparegui el caràcter a. Demostreu que, per a qualsevol símbol $a \in \Sigma$, la seva profunditat en un arbre prefix que produeix un codi de Huffman òptim és $O(\lg \frac{1}{p_a})$. (Ajuts: en un arbre prefix que s'utilitzi per a dissenyar el codi Huffman, la probabilitat d'un nus és la suma de les probabilitats dels fills. La probabilitat de l'arrel és, doncs, 1.)

49. Una tribu de matemàtics ha decidit utilitzar un alfabet molt concís de 3 símbols (més el blanc $\flat$) per a comunicar-se entre ells. Utilitzant cadenes dels símbols $\{+, -, \star\}$ els matemàtics codifiquen totes els paraules que necessiten per a comunicar-se. Donada la seqüència $+ - - + \star\flat \star + \star - \star \flat\star$, trobeu la freqüència de cada un dels símbols $\{+, -, \star, \flat\}$. Dibuixeu l'arbre de Huffman. Quina és la compressió màxima que podeu obtenir utilitzant Huffman? Quin és el guany de compressió respecte a utilitzar una compressió de longitud fixada? (i.e. utilitzant $\{0, 1\}^2$). Tingueu en compte que els matemàtics es comuniquen entre si utilitzant Internet, per tant, abans d'enviar el missatges, l'ordinador converteix els símbols del seu alfabet en cadenes binàries via ASCII.

50. Sigui $T = (V, E)$ un arbre no dirigit amb n vèrtexs. Per a $u, v \in V$, sigui $d(u, v)$ el nombre d 'arestes del camí de u a v en T. Definim el *centre* de T com el vèrtex

$$c = argmin_{v \in V}\{max_{u \in V} d(u, v)\}.$$

Describiu un algorisme de cost $O(n)$ per a obtenir un centre de T.

51. CinemaVis ha de programar l'aparició d'un seguit d'anuncis a una pantalla gegant a la Plaça del Mig la diada de Sant Jordi. La tirada d'anuncis es pot iniciar a temps 0 (l'inici programat) però mai abans. A més, CinemaVis disposa d'un conjunt de n anuncis per fer la selecció. L'anunci i té una durada de 1 minut i té associat dos valors reals no negatius t_i i b_i. L'anunciat pagarà b_i euros a CinemaVis si l'anunci i s'emet a l'interval $[0, t_i]$ i 0 euros si s'emet després. Cap dels n anuncis es pot mostrar més d'una vegada. CinemaVis vol projectar la selecció d'anuncis que li proporcioni màxim benefici. Dissenyeu un algorisme, el més eficient que podeu, per a resoldre aquest problema.

52. *(Agenda)* A la vostra agenda teniu una llista L de totes les tasques que heu de completar en el dia de avui. Per a cada tasca $i \in L$ s'especifica la durada $d_i \in \mathbb{N}$ que indica el temps necessari per a completar-la i un factor de penalització $p_i \in \mathbb{Z}^+$ que n'agreuja el retard. Heu de determinar en quin ordre realitzar totes les tasques per obtenir el resultat que menys penalització total acumuli.

Tingueu en compte que:

- en un instant de temps només podeu realitzar una única tasca,
- una vegada comenceu a fer una tasca, heu de continuar-la fins a finalitzar-la, i
- s'han de completar totes les tasques.

El criteri d'optimització és la penalització total que s'acumula. La penalització real associada a una tasca $i \in L$ és el temps de finalització t_i de la seva realització, multiplicat per la seva penalització p_i. El temps de finalització t_i es correspon al temps transcorregut des de l'inici de la jornada laboral (és a dir, des de l'instant de temps 0) fins al moment en que s'ha finalitzat la tasca.[1]

Considereu l'algorisme voraç que programa les tasques en ordre decreixent de factor de penalització p_i. Determineu si aquest algoritme resol el problema. En cas que no ho faci, proporcioneu un algorisme (tant eficient com pogueu) per resoldre'l.

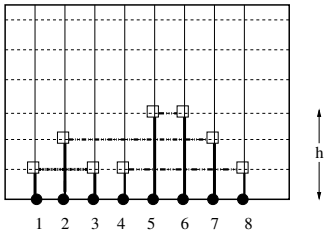
[1] Observeu que, segons les restriccions del problema, la realització d'una tasca $i \in L$ amb temps de finalització t_i haurà començat a l'instant $t_i - d_i$.

53. Sigui $X = \{(a_1, b_1), \ldots, (a_n, b_n)\}$ un conjunt de n intervals. Diem que un conjunt de punts $P = \{p_1, \ldots, p_k\}$ *intercepta* un conjunt d'intervals X si tot interval a X conté com a mínim un punt de P, és a dir,

$$\forall_{1 \leq i \leq n} \, \exists_{1 \leq j \leq k} \quad a_i \leq p_j \leq b_i.$$

Describiu i analitzeu un algorisme eficient per, donats el conjunt d'intervals X i el conjunt de punts P, calculi el subconjunt més petit de punts que intercepta X.

54. Als circuits VLSI, s'utilitza un encaminament Manhattan sobre la placa aïllant on va muntat el circuit. Les connexions horitzontals van per la cara de sota i les connexions verticals per la cara de sobre. Quan es necessiten connectar les connexions horitzontals amb les verticals, es perfora la placa amb el que s'anomena una *via*. Les connexions del circuit amb l'exterior es realitzen amb *pins* (veure la figura, on les connexions de sobre estan dibuixades en sòlid i les que van per sota amb línia discontínua). Tant les connexions horitzontals com verticals segueixen unes pistes dibuixades sobre la placa en forma d'una graella, i els pins estat alineats a un extrem de la placa. Sigui h el nombre de pistes horitzontals utilitzades. Si $L = \{(p_1, q_1), (p_2, q_2), \ldots, (p_n, q_n)\}$ són una seqüència de parells de pins a connectar (dos a dos), volem dissenyar un algorisme que connecti els parells de pins utilitzant el mínim nombre de pistes horitzontals h. Per exemple, considereu la següent figura:



Tenim com a entrada $L = \{(1, 3), (2, 7), (4, 8), (5, 6)\}$, el nombre de h és 3, i no es pot fer amb $h = 2$.

En particular: dissenyeu un algorisme eficient, que donats n parells de pins, resolgui el problema de l'encaminament, de manera que es minimitzi h. Doneu-ne la complexitat i demostreu-ne la correctesa.

55. Doneu un algorisme que resolgui el següent problema i doneu la seva complexitat en funció de n. Justifiqueu-ne la correctesa:

Volem anar de Barcelona a Paris seguint l'autopista a través de Lyon, i tenim n benzineres, $B_1 \ldots, B_n$, al llarg d'aquesta ruta. Amb el diposit ple, el vostre cotxe pot funcionar K km. La benzinera B_1 és a Barcelona, i cada B_i, $2 \leq i \leq n$ és a $d_i < K$ km. de la benzinera B_{i-1}. La benzinera B_n és a Paris. Quina és l'estratègia per aturar-se el mínim nombre de cops al llarg del viatge?

Una solució: L'estratègia que farem servir és esperar el màxim per a carregar benzina, però sense quedar-nos amb el dipòsit buit. Aquest criteri es pot implementar com el següent algorisme voraç:

```
function GREEDY(d_1, ... d_n, K)
    d = 0;  R = ∅;  j = 1
    for i = 1 ... n − 1 do
        if d + d_{i+1} > K then
            j := j + 1;
            R = R ∪ {B_j};
            d = 0;
        end if
        d := d + d_{i+1};
    end for
    return (j);
end function
```

Per demostrar la correctesa de l'algorisme compararem la solució obtinguda pel nostre algorisme amb una posible solució òptima amb menys aturades. Sigui $R = \{b_1, \ldots, b_j\}$ les benzineres seleccionades per GREEDY. Suposem que la solució òptima requereix $m < j$ aturades, sigui $S = \{c_1, \ldots, c_m\}$ el conjunt de les benzineres a una solució òptima.

Primer demostrarem que, per $j = 1, \ldots, m$, $d(B_1, b_i) \geq d(B_1, c_i)$. L'enunciat és cert per $j = 1$, si no ens quedaríem sense benzina abans d'arribar a c_1. Suposem que l'enunciat és cert per $i < j$. Llavors tenim

$$d(B_1, c_j) - d(B_1, c_{j-1}) \leq K,$$

ja que S és una solució, i

$$d(B_1, c_j) - d(B_1, b_{j-1}) \leq d(B_1, c_j) - d(B_1, c_{j-1}),$$

per hipòtesi d'inducció. Combinant les dues desigualtats tenim

$$d(B_1, c_j) - d(B_1, b_{j-1}) \leq K.$$

Llavors, c_j ha de ser abans de b_j.

Com a conseqüència del resultat tenim que $d(b_m, B_n) \geq d(c_m, B_n) \leq K$, llavors l'algorisme voraç no s'aturaria a cap benzinera després de b_m i tenim $j = m$.

Per tant GREEDY és òptim i la seva complexitat és $O(n)$.